To use aggregation data:

```
# Use aggregations to analyze the data.
agg_query = {
    'size': 4,
    'aggs': {
        'popular_tags': {
            'terms': {
                'field': 'tags',
                'size': 10
            }
        },
        'avg_rating': {
            'avg': {
                'field': 'rating'
            }
        },
        'monthly_posts': {
            'date_histogram': {
                'field': 'publish_date',
                'calendar_interval': 'month'
            }
        }
    }
}

response = es.search(index='articles', body=agg_query)
print(f"Average rating: {response['aggregations']['avg_rating']['value']}")
print("\t\t\t", "="*20)

for bucket in response['aggregations']['popular_tags']['buckets']:
    print(f"Tag: {bucket['key']}, Count: {bucket['doc_count']}")
print("\t\t\t", "="*20)

for bucket in response['aggregations']['monthly_posts']['buckets']:
    print(f"Monthly Posts: {bucket['key']}, Count: {bucket['doc_count']}")
print("\t\t\t", "="*20)
```

==========

- terms: like SQL GROUP BY
- avg: returns the average value of a field
- date_histogram: buckets data by time intervals, useful for trend analysis (like in the statistics book)

==========

Document Size Limits

Elasticsearch supports documents up to 2GB, but in practice, large documents impact performance.

- Large documents consume more memory during indexing and querying
- Text analysis and term indexing become slower
- It's better to split large documents into smaller parts

==========

Mapping Constraints

Elasticsearch has limits around how documents can be structured:

- Max field name length: 1000 characters
- Max nested object depth: 20
- Max number of fields per index (default): 1000

If you're generating dynamic fields or deeply nested JSON, these limits can cause failures.

==========

Memory Usage and High-Cardinality Fields

Elasticsearch loads field data into memory for sorting and aggregations. High-cardinality fields (with many unique values) use more memory.

- Sorting or aggregating on fields like user_id or email can trigger memory limits
- Circuit breakers are used to stop requests that could crash the cluster

To avoid issues:

- Avoid sorting or aggregating on high-cardinality fields
- Use doc_values: false where aggregation is not needed
- Monitor heap usage regularly

==========

Timeout Settings

Long-running queries can hang and block resources. Set timeouts to avoid this.

Global client-level timeout:

```
es = Elasticsearch(
    ['localhost:9200'],
    timeout=30,
    max_retries=3,
    retry_on_timeout=True
)
```

Per-query timeout:

```
query = {
    'query': {'match_all': {}},
    'timeout': '10s'
}
response = es.search(index='articles', body=query)
```

Set appropriate timeouts based on query complexity.

==========

Indexing Performance and Refresh Intervals

By default, Elasticsearch refreshes the index every second, which makes documents searchable. For bulk operations, you can disable refresh temporarily to improve performance.

```
# Turn off refresh
es.indices.put_settings(
    index='articles',
    body={'index': {'refresh_interval': '-1'}}
)

# Perform bulk indexing
bulk(es, document_list, chunk_size=1000)

# Restore refresh interval
es.indices.put_settings(
    index='articles',
    body={'index': {'refresh_interval': '30s'}}
)

# Manually refresh if needed
es.indices.refresh(index='articles')
```

==========

Mapping Conflicts

Field types in Elasticsearch are fixed once assigned. For example, if a field is first indexed as a number, later attempts to index it as a string will fail.

```
def safe_index_document(index, doc_id, document):
    try:
        return es.index(index=index, id=doc_id, body=document)
    except Exception as e:
        if 'mapper_parsing_exception' in str(e):
            print(f"Mapping conflict for document {doc_id}: {e}")
            return None
        else:
            raise

# Example usage
doc1 = {'price': 29.99}         # Float
doc2 = {'price': 'twenty'}      # String

safe_index_document('products', 1, doc1)
safe_index_document('products', 2, doc2)  # Triggers conflict
```

Use explicit mappings where possible to avoid type guessing.

==========

Resource Exhaustion

Expensive queries, large aggregations, and wildcard searches can exhaust system resources if not managed carefully.

- Monitor node heap memory
- Avoid unbounded wildcard or regex filters
- Watch for unassigned or relocating shards

```python
def check_cluster_health():
    health = es.cluster.health()
    stats = es.nodes.stats()

    print(f"Cluster status: {health['status']}")

    for node_id, node in stats['nodes'].items():
        heap_used = node['jvm']['mem']['heap_used_percent']
        print(f"Node {node_id} heap usage: {heap_used}%")
        if heap_used > 85:
            print(f"Warning: High heap usage on node {node_id}")

    return health['status'] == 'green'
```

==========

These limits aren't blockers, but you need to design around them. Define clear mappings, set timeouts, and monitor resource usage early in development to avoid costly surprises later.

==========

```python
# Index a document
doc = {
    'title': 'Elasticsearch Python Integration',
    'content': 'Complete guide to using Elasticsearch with Python',
    'timestamp': '2024-01-15T10:30:00'
}

response = es.index(index='guides', body=doc)
doc_id = response['_id']

# Retrieve the same document
retrieved = es.get(index='guides', id=doc_id)
print(f"Title: {retrieved['_source']['title']}")

# Update it
es.update(
    index='guides',
    id=doc_id,
    body={'doc': {
        'content': 'Updated content with more details',
        'last_modified': '2024-01-16T14:22:00'
    }}
)

# Delete it
es.delete(index='guides', id=doc_id)
```

==========

```python
success, failed = bulk(es, actions, chunk_size=1000)
print(f"Indexed: {success} documents")

if failed:
    print(f"Failed to index {len(failed)} documents")
    for item in failed:
        print(f"Error: {item}")
```

==========

Scroll API for Large Search Results

If you need to retrieve more than 10,000 results (default limit), use the Scroll API to paginate through large datasets.

```
from elasticsearch.helpers import scan

def process_large_dataset(index_name):
    total = 0

    for doc in scan(es, index=index_name, query={'query': {'match_all': {}}},
    size=1000, scroll='2m'):
        process(doc['_source'])
        total += 1

    print(f"Processed {total} documents")

def process(doc):
    # Replace this with your logic
    pass

process_large_dataset('large_index')
```

==========

Real-Time Indexing and Search Behavior

Elasticsearch is near real-time; indexed documents become searchable after the next refresh. The default refresh interval is 1s.

To make a document available for immediate search, you can either:

1. Manually refresh the index
2. Use the `refresh='wait_for'` parameter

```
# Manual refresh example
es.indices.refresh(index='realtime_demo')

# Index with immediate search availability
es.index(
    index='alerts',
    body={
        'alert_type': 'critical',
        'message': 'System failure detected',
        'timestamp': '2024-01-15T10:30:00'
    },
    refresh='wait_for'
)

# Now this will find the document
res = es.search(index='alerts', body={'query': {'term': {'alert_type':
'critical'}}})
print(f"Found: {res['hits']['total']['value']} critical alerts")
```

==========

Use `elasticsearch-dsl` for Query Building

The `elasticsearch-dsl` library is a higher-level abstraction over `elasticsearch-py` that provides a fluent way to build queries and aggregations. It's useful for complex queries and readable logic.

```
from elasticsearch_dsl import Search, Q
from elasticsearch_dsl.connections import connections

# Connect DSL to the cluster
connections.create_connection(hosts=['localhost:9200'])

# Build a search query
s = Search(index='articles')
s = s.query('match', title='elasticsearch')
s = s.filter('term', status='published')

# Run query
res = s.execute()
for hit in res:
    print(hit.title)
```